

Architecture

Lecture canonique du projet

Le projet suit une logique de montée en maturité continue :

- **Bloc 2** : pipeline data exécutable localement, ingestion, Bronze, Silver, chargement SQL local et API FastAPI ;
- **Bloc 3** : ajout d'une vraie couche **Gold DWH** en schéma en étoile, tout en conservant `gold_daily_features` comme table de restitution ;
- **Bloc 4** : formalisation du même socle sous forme de **Data Lake gouverné**.

Couches

- **Bronze** : données brutes, traçabilité, rejouabilité
- **Silver** : données nettoyées, normalisées et enrichies
- **Gold** :
 - tables analytiques / décisionnelles (DWH)
 - dataset de service `gold_daily_features` pour restitution / API

Mapping Bloc 4

- Bronze -> Raw
- Silver -> Curated
- Gold -> Consumption

Implémentation locale et cible

- stockage local par défaut via `LocalStorageBackend`
- compatibilité GCS préparée via `GCSStorageBackend`
- couche SQL locale démontrable via `SQLite`
- cible analytique avancée documentée via `Delta Lake`

Eco-responsabilite (C3 / RGENS)

La stratégie d'eco-conception du projet s'appuie sur les principes du Référentiel Général d'Eco-conception des Services Numériques (RGENS) :

- 1 **Sobriété du stockage** : seules les données agrégées au niveau régional sont collectées. Pas de données individuelles, pas de logs excessifs. La politique de retention supprime automatiquement les données Silver après 5 ans et Gold après 2 ans.
- 2 **Sobriété du traitement** : SQLite en local pour la demo et le développement. Pas de cluster Spark ou Hadoop pour traiter quelques milliers de lignes. La migration vers Delta Lake / Spark est documentée mais ne sera activée qu'au passage à l'échelle (> 1M lignes).
- 3 **Sobriété de l'infrastructure** : pas de services cloud actifs en permanence. Le StorageBackend est une abstraction qui permet de basculer vers GCS uniquement quand le volume le justifie.
- 4 **Minimisation des transferts** : les données sont transformées localement (Bronze -> Silver -> Gold sur la même machine). L'API REST n'expose que les données nécessaires, filtrées par date et région.
- 5 **Choix des outils** : Python (léger), FastAPI (async, performant), SQLite (zero serveur), Airflow

(batch quotidien vs streaming permanent). Chaque outil est dimensionné au besoin réel, pas au besoin hypothétique.

- 6 **Choix des prestataires** : Apache Atlas (open source, on-premise) plutôt qu'un service cloud avec empreinte carbone liée aux data centers.

Principe de maintenance

Toujours distinguer :

- ce qui est **implémenté**
- ce qui est **documenté / préparé**
- ce qui est **proposé pour la suite**